



1. For Loop (3 Questions)

1. Write a program to print the multiplication table of a given number using a for loop.

```
num = 78

for i in range(1, 11):
    print(num, "*", i, "=", num * i)
```

```
78 * 1 = 78
78 * 2 = 156
78 * 3 = 234
78 * 4 = 312
78 * 5 = 390
78 * 6 = 468
78 * 7 = 546
78 * 8 = 624
78 * 9 = 702
78 * 10 = 780
```



2. Write a program to print all prime numbers between 1 and 100 using a for loop.

```
for num in range(1, 100):
    is_prime = True

    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break

    if is_prime:
        print(num)
```

```
1
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
```



```
53
59
61
67
71
73
79
83
89
97
101
```

3. Write a program to find the sum of all even numbers from 1 to n using a for loop.

```
n = int(input("Enter the value of n: "))
sum = 0
for i in range(2, n + 1, 2):
    sum = sum + i
print("Sum of even numbers =", sum)
Enter the value of n: 29
Sum of even numbers = 210
```



2. While Loop (3 Questions)

1. Write a program to reverse a given number using a while loop.

```
num = int(input("Enter a number: "))
reverse = 0
while num > 0:

    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10
print("Reversed number is:", reverse)
Enter a number: 9075
Reversed number is: 5709
```



2. Write a program to check whether a given number is a palindrome using a while loop.

```
num = int(input("Enter a number: "))
original = num
reverse = 0
while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num = num // 10

if original == reverse:
    print(original, "is a Palindrome")
else:
    print(original, "is not a Palindrome")
Enter a number: 300
300 is not a Palindrome
```



3. Write a program to calculate the sum of digits of a number using a while loop.

```
num = int(input("Enter a number: "))
sum_digits = 0
while num > 0:
    digit = num % 10
    sum_digits = sum_digits + digit
    num = num // 10
print("Sum of digits =", sum_digits)
Enter a number: 567
Sum of digits = 18
```

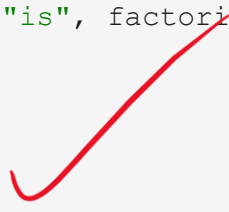


3. Functions (5 Questions)

1. Write a function to calculate the factorial of a number.

```
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
    return fact
num = int(input("Enter a number: "))
if num < 0:
    print("Factorial is not defined for negative numbers.")
else:
    print("Factorial of", num, "is", factorial(num))
factorial(6)
```

720

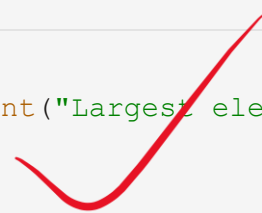


2. Write a function that takes a list as input and returns the largest element.

```
def largest_element(numbers):
    largest = numbers[0]
    for num in numbers:
        if num > largest:
            largest = num
    return largest

my_list = [125678, 45, 790, 908, 2300]
print("Largest element:", largest_element(my_list))
```

Largest element: 125678




3. Write a function to check whether a given string is a palindrome.

```
def is_palindrome(text):
    text = text.lower()
    if text == text[::-1]:
        return True
    else:
        return False

word = input("Enter a string: ")
if is_palindrome(word):
    print("It is a Palindrome")
else:
    print("It is not a palindrome")
```

Enter a string: 6482
It is not a palindrome




4. What is a function in Python? Explain its advantages.

A function in Python is a block of reusable code that performs a specific task.

Advantages:-

5. Differentiate between user-defined functions and built-in functions with examples.

4. OOPS (Theory – 9 Questions)

1. What is object Oriented Programming (OOP)?

Object-Oriented Programming (OOP) is a programming paradigm that organizes a program using objects and classes.

2. Explain the difference between Class and Object with an example.

Class	Object
A class is a blueprint or template for creating objects.	An object is an instance of a class .

3. What are the four pillars of OOP? Explain each.

1. Encapsulation: Encapsulation is the process of combining data (variables) and methods (functions) into a single unit (class).

2. Polymorphism: Polymorphism means one interface, many forms.

3. Abstraction: Abstraction means hiding unnecessary implementation details and showing only the essential features.

4. Inheritance: Inheritance allows one class to inherit the properties and methods of another class.

4. What is Inheritance? Explain its types.

Inheritance

Inheritance allows one class to inherit the properties and methods of another class.

1. Single Inheritance: One child class inherits from one parent class.

2. Multiple Inheritance: One child class inherits from more than one parent class.

3. Multilevel Inheritance: A class inherits from another derived class.

4. Hierarchical Inheritance: Multiple child classes inherit from the same parent class.

5. Hybrid Inheritance: A combination of two or more types of inheritance (such as multiple and multilevel inheritance).

5. What is Polymorphism? Explain Method Overriding and Method Overloading.

Polymorphism: Polymorphism means one interface, many forms.

1.Method Overriding: Method Overriding occurs when a child class provides its own implementation of a method that is already defined in the parent class.

2.Method Overloading: Method Overloading means defining multiple methods with the same name but with different numbers or types of parameters.

6.What is Encapsulation? What are its advantages?

Encapsulation: Encapsulation is the process of combining data (variables) and methods (functions) into a single unit (class).

Advantages:-

7.What is Abstraction? How is it implemented in Python?

Abstraction: Abstraction means hiding unnecessary implementation details and showing only the essential features.

In Python, abstraction is implemented using the abc (Abstract Base Class) module.

8.What is the difference between Abstraction and Encapsulation?

Abstraction	Encapsulation
 Abstraction means hiding implementation details and showing only the essential features.	Encapsulation means wrapping data and methods into a single unit (class) and protecting the data.
 Achieved using abstract classes and abstract methods (ABC module).	Achieved using classes and private members (using __).

9.Explain the use of the super() function in Python.

The super() function is used to call methods or constructors of the parent class from a child class.

It is commonly used with inheritance to reuse the parent class's code without rewriting it.